

QtSpim Assembly Code Format Guide

ES 336 – Computer Organization and Architecture

Indian Institute of Technology Gandhinagar

What is the Expected Format of Input Assembly Code?

To be parseable by QtSpim, your assembly code must follow the standard MIPS structure. The file should be a plain text file, typically saved with the `.s` or `.asm` extension.

Download QtSpim Simulator:

You can download QtSpim from the official SourceForge repository:

<https://sourceforge.net/projects/spimsimulator/files/>

Here is the breakdown of the expected format.

1 The Basic Skeleton

Every runnable QtSpim program generally needs two main sections: **Data** (variables) and **Text** (code).

```
1 # 1. DATA SEGMENT: Define variables and constants here
2 .data
3     variable_name: .type    value        # e.g., msg: .asciiz "
4         Hello"
5 # 2. TEXT SEGMENT: Actual assembly instructions go here
6 .text
7     .globl main                # make 'main'
8                                 accessible to the simulator
9 main:                          # Program entry point
10    # ... your code here ...
11
12    # Exit sequence (Crucial!)
13    li $v0, 10
14    syscall
```

2 Key Syntax Rules

2.1 Comments

- Start with a hash symbol (#). Anything after # on a line is ignored.
- Example: `add $t0, $t1, $t2 # This adds t1 and t2`

2.2 Directives (The “Dot” Commands)

Directives tell the assembler *how* to process the data. They always start with a dot.

- `.data`: Marks the start of the data segment.
- `.text`: Marks the start of the code segment.
- `.globl main`: Declares the label `main` as global so QtSpim knows where to start execution.

2.3 Data Types (in `.data`)

- `.asciiz "string"`: Stores a string ending with a null character (standard for printing).
- `.ascii "string"`: Stores a string *without* the null character.
- `.word 5, 10, 15`: Stores 32-bit integers.
- `.space n`: Reserves `n` bytes of empty memory (useful for arrays).
- `.byte 1`: Stores a single 8-bit byte.

2.4 Labels

- Used to name memory locations (variables) or code destinations (functions/loops).
- Must end with a colon (:).
- Examples: `loop_start:`, `my_variable:`, `main:`.

2.5 Registers

QtSpim accepts both register **names** (recommended) and **numbers**.

- **Names**: `$t0`, `$a0`, `$sp`, `$ra` (easier to read).
- **Numbers**: `$8`, `$4`, `$29`, `$31`.

3 The System Call (“The OS Interface”)

QtSpim simulates an Operating System. To do Input/Output or Exit, you must use `syscall`.

The Pattern:

1. Load the **Service Code** into register `$v0`.
2. Load **Arguments** (if any) into registers `$a0`, `$a1`, etc.
3. Execute `syscall`.

Service	Code (\$v0)	Arguments (\$a0)	Result
Print Integer	1	Integer to print	Output to console
Print String	4	Address of string	Output to console
Read Integer	5	(None)	Input in \$v0
Read String	8	Address of buffer, Length	Input in buffer
Exit Program	10	(None)	Stops simulator

Table 1: Common QtSpim System Calls

4 Common Formatting Mistakes

1. **Missing the Exit Call:** If you don't put `li $v0, 10` and `syscall` at the end, QtSpim will “fall off the edge” of your memory and crash or try to execute random data as code.
2. **Case Sensitivity:** MIPS instructions (`add`, `lw`) are generally case-insensitive in QtSpim, but **Labels** are Case-Sensitive. `Loop:` is different from `loop:`.

5 A Complete Parseable Template

You can copy-paste this directly into a `.s` file to test.

```

1  # QtSpim Template
2  # Defines an array, sums it, and prints the result.
3
4  .data
5      # Define an array of 3 words (integers): 10, 20, 30
6      my_array:    .word 10, 20, 30
7      length:      .word 3
8      msg:         .asciiz "The sum is: "
9
10 .text
11     .globl main
12
13 main:

```

```
14      # 1. Initialize registers
15      la $t0, my_array          # Load address of array into $t0
16      lw $t1, length           # Load array length into $t1
17      li $t2, 0                 # $t2 will be our accumulator (sum
                                # = 0)
18      li $t3, 0                 # $t3 is our loop counter (i = 0)
19
20  loop_start:
21      # 2. Check Loop Condition (if i == length, exit)
22      beq $t3, $t1, loop_end
23
24      # 3. Load the current array element
25      lw $t4, 0($t0)           # Load word at address $t0
26
27      # 4. Add to the sum
28      add $t2, $t2, $t4        # sum = sum + array[i]
29
30      # 5. Increment pointers and counters
31      addi $t0, $t0, 4          # Move array pointer forward by 4
                                # bytes
32      addi $t3, $t3, 1          # i++
33      j loop_start             # Jump back to start of loop
34
35  loop_end:
36      # 6. Print Message
37      li $v0, 4                 # Syscall 4: Print String
38      la $a0, msg
39      syscall
40
41      # 7. Print Sum
42      li $v0, 1                 # Syscall 1: Print Integer
43      move $a0, $t2             # Move sum to argument register
44      syscall
45
46      # 8. Exit Cleanly
47      li $v0, 10                # Syscall 10: Exit
48      syscall
```

Note: Always test your code in QtSpim to ensure proper syntax and logic. Use the QtSpim console to debug any errors that may occur during execution.